

School of Computer Science and Artificial Intelligence

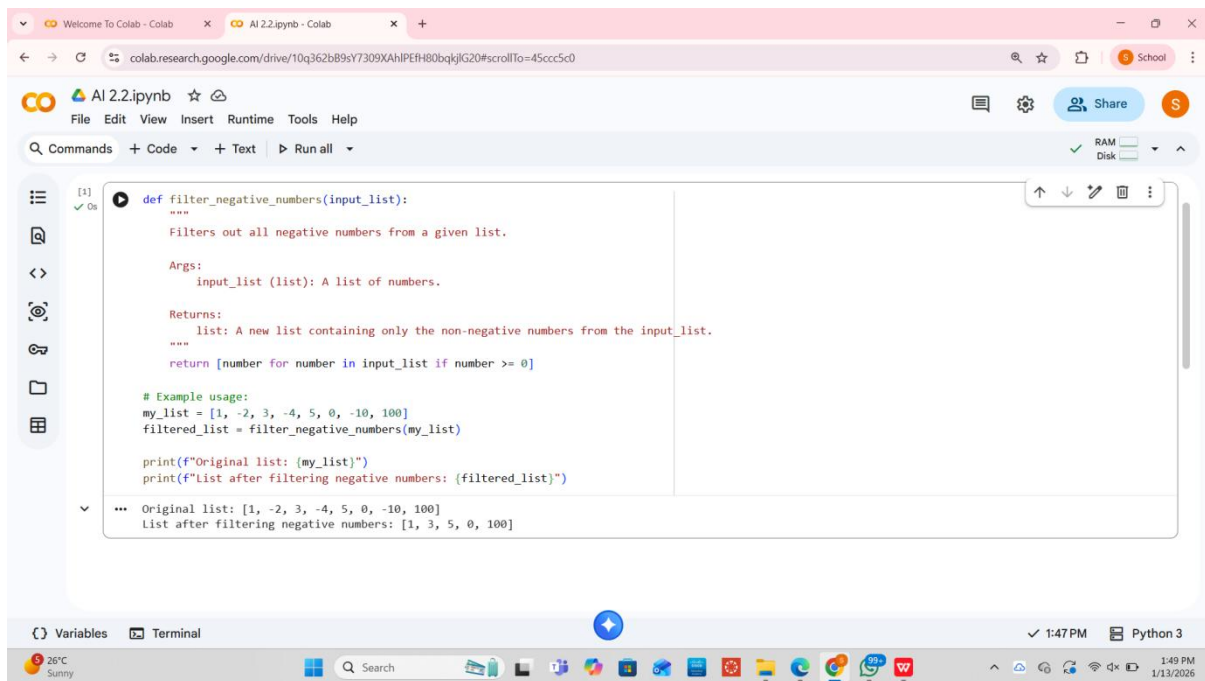
Lab Assignment 2.2

Program : B. Tech (CSE)
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : III
Academic Session : 2025-2026
Name of Student : Sankeerthana
Enrollment No. : 2403A51L41
Batch No. : 52
Date : 13-01-2026

Task 1

Prompt: Write a python program to that filters out all negative numbers from a list using function

Code:



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: `colab.research.google.com/drive/10q362b89sY7309XAHlPEFH80bqkjlG20#scrollTo=45ccc5c0`. The notebook is titled "AI 2.2.ipynb". The code editor contains the following Python code:

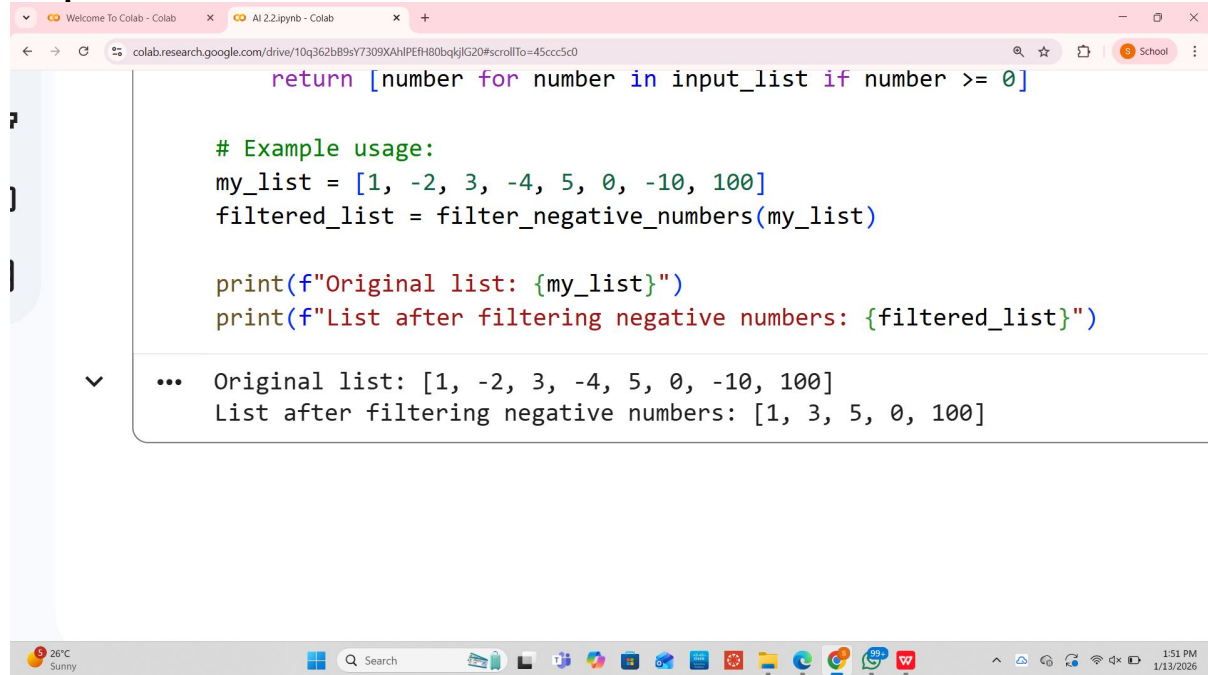
```
def filter_negative_numbers(input_list):  
    """  
    Filters out all negative numbers from a given list.  
  
    Args:  
        input_list (list): A list of numbers.  
  
    Returns:  
        list: A new list containing only the non-negative numbers from the input_list.  
    """  
    return [number for number in input_list if number >= 0]  
  
# Example usage:  
my_list = [1, -2, 3, -4, 5, 0, -10, 100]  
filtered_list = filter_negative_numbers(my_list)  
  
print(f"Original list: {my_list}")  
print(f"List after filtering negative numbers: {filtered_list}")
```

The output of the code is displayed at the bottom of the notebook:

```
Original list: [1, -2, 3, -4, 5, 0, -10, 100]  
List after filtering negative numbers: [1, 3, 5, 0, 100]
```

The bottom of the image shows a Windows taskbar with the system clock indicating 1:47 PM on 1/13/2026, and the Python 3 environment is active.

Output:



```
return [number for number in input_list if number >= 0]

# Example usage:
my_list = [1, -2, 3, -4, 5, 0, -10, 100]
filtered_list = filter_negative_numbers(my_list)

print(f"Original list: {my_list}")
print(f"List after filtering negative numbers: {filtered_list}")

... Original list: [1, -2, 3, -4, 5, 0, -10, 100]
List after filtering negative numbers: [1, 3, 5, 0, 100]
```

Explanation:

The function `filter_negative_numbers` is used to remove all negative numbers from a given list and return a new list containing only non-negative values.

The function takes one parameter called `input_list`, which is a list of numbers. Inside the function, a list comprehension is used to iterate through each element in the list. During this iteration, a condition checks whether the number is greater than or equal to zero. If the condition is true, the number is included in the new list; otherwise, it is ignored.

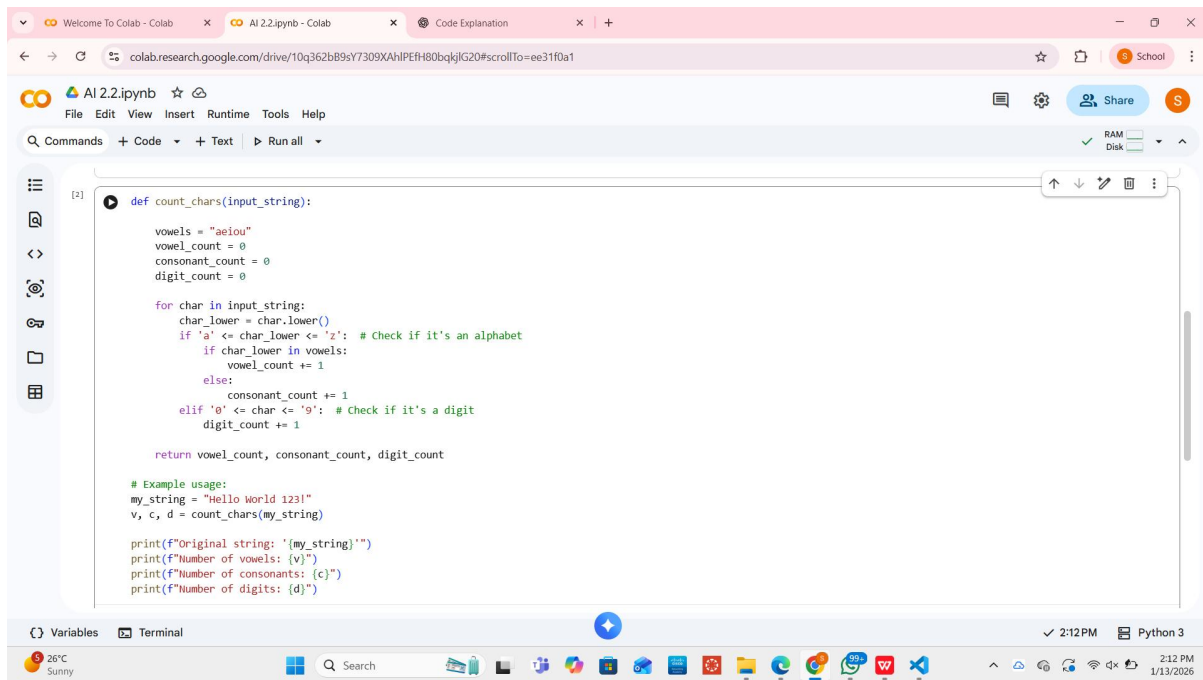
This approach ensures that all negative numbers are filtered out while positive numbers and zero are retained. The function then returns the newly created list containing only non-negative numbers.

In the example, the original list contains both positive and negative numbers. After calling the function, the resulting list includes only the positive numbers and zero, while all negative values are removed.

Task2:

Prompt: Write a python program that counts vowels, consonants, and digits in a string.

Code:



```
def count_chars(input_string):
    vowels = "aeiou"
    vowel_count = 0
    consonant_count = 0
    digit_count = 0

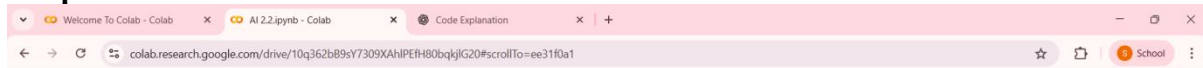
    for char in input_string:
        char_lower = char.lower()
        if 'a' <= char_lower <= 'z': # Check if it's an alphabet
            if char_lower in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
        elif '0' <= char <= '9': # Check if it's a digit
            digit_count += 1

    return vowel_count, consonant_count, digit_count

# Example usage:
my_string = "Hello World 123!"
v, c, d = count_chars(my_string)

print(f"Original string: '{my_string}'")
print(f"Number of vowels: {v}")
print(f"Number of consonants: {c}")
print(f"Number of digits: {d}")
```

Output:



```
print(f"Number of digits: {d}")
```

- Original string: 'Hello World 123!'
- Number of vowels: 3
- Number of consonants: 7
- Number of digits: 3

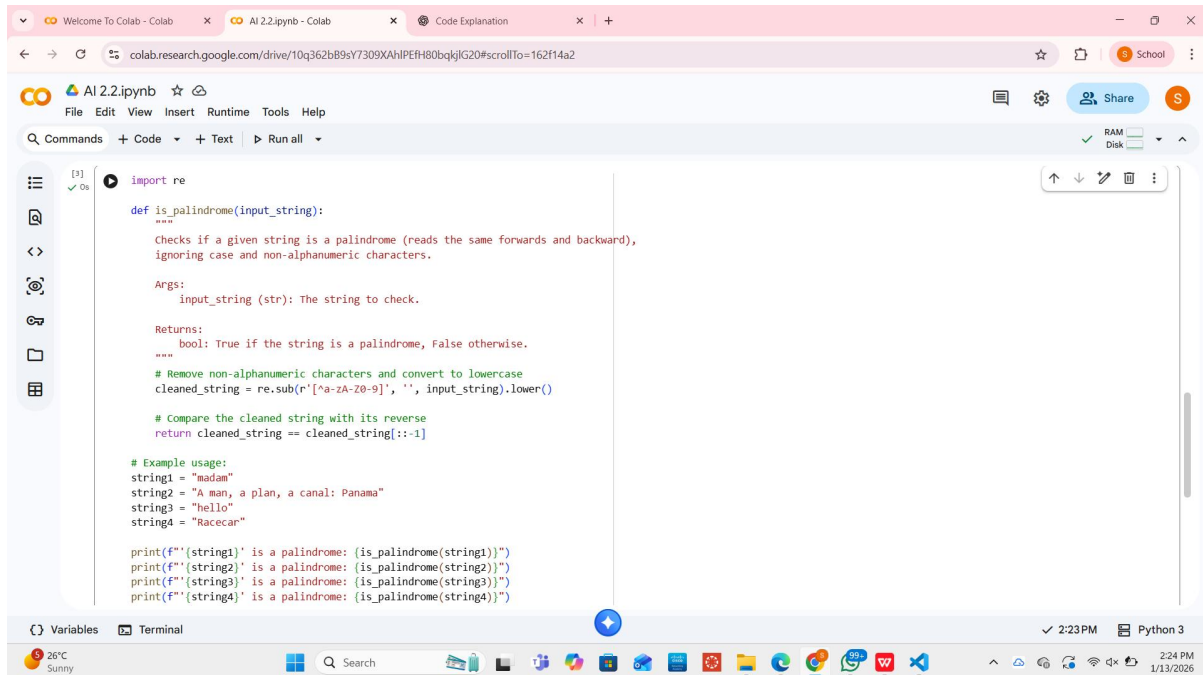
Explantation:

The function `count_chars` counts the number of vowels, consonants, and digits in a given string. It checks each character in the string one by one. If the character is an alphabet, it is classified as a vowel or a consonant. If the character is a digit, it is counted as a digit. Spaces and special characters are ignored. Finally, the function returns the total count of vowels, consonants, and digits.

Task3

Prompt: Write a python program that checks the string is palindrome are not using functions

Code:
Gemini:



The screenshot shows a Google Colab notebook with a single code cell. The code defines a function `is_palindrome` that takes an `input_string` and returns a boolean. It uses `re.sub` to remove non-alphanumeric characters and `lower()` to convert the string to lowercase. It then compares the string with its reverse. Example strings are provided and tested.

```
import re

def is_palindrome(input_string):
    """
    Checks if a given string is a palindrome (reads the same forwards and backward),
    ignoring case and non-alphanumeric characters.

    Args:
        input_string (str): The string to check.

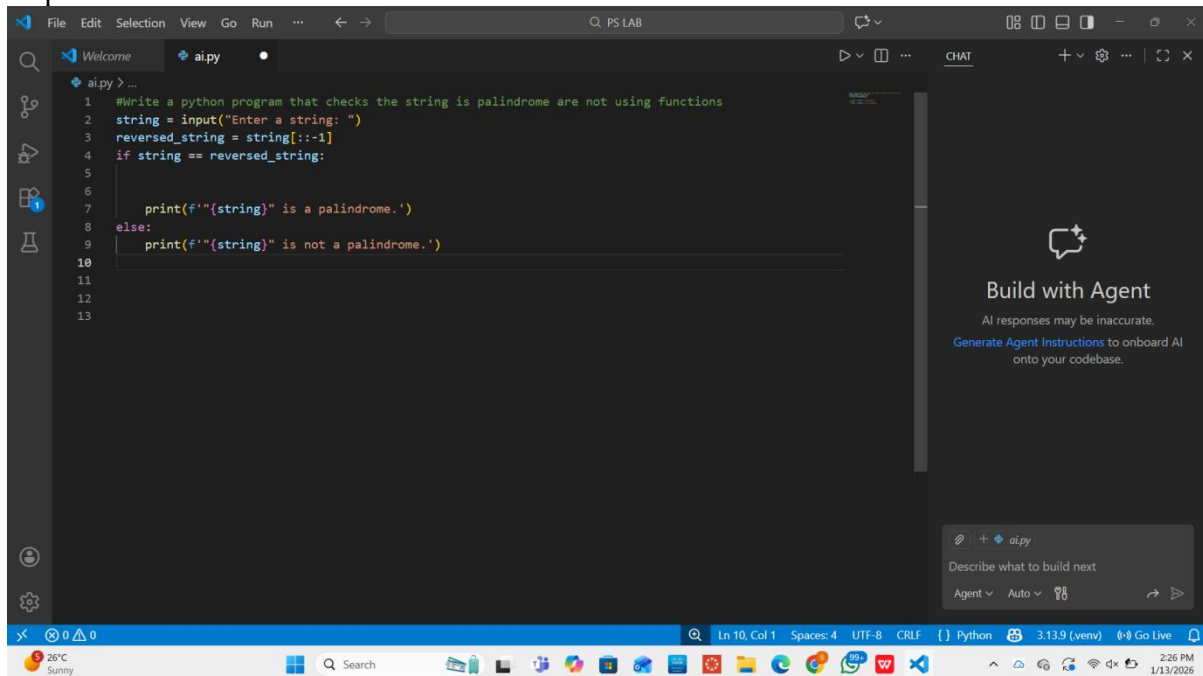
    Returns:
        bool: True if the string is a palindrome, False otherwise.
    """
    # Remove non-alphanumeric characters and convert to lowercase
    cleaned_string = re.sub(r'[^a-zA-Z0-9]', '', input_string).lower()

    # Compare the cleaned string with its reverse
    return cleaned_string == cleaned_string[::-1]

# Example usage:
string1 = "madam"
string2 = "A man, a plan, a canal: Panama"
string3 = "hello"
string4 = "Racecar"

print(f'{string1} is a palindrome: {is_palindrome(string1)}')
print(f'{string2} is a palindrome: {is_palindrome(string2)}')
print(f'{string3} is a palindrome: {is_palindrome(string3)}')
print(f'{string4} is a palindrome: {is_palindrome(string4)}')
```

Copilot:

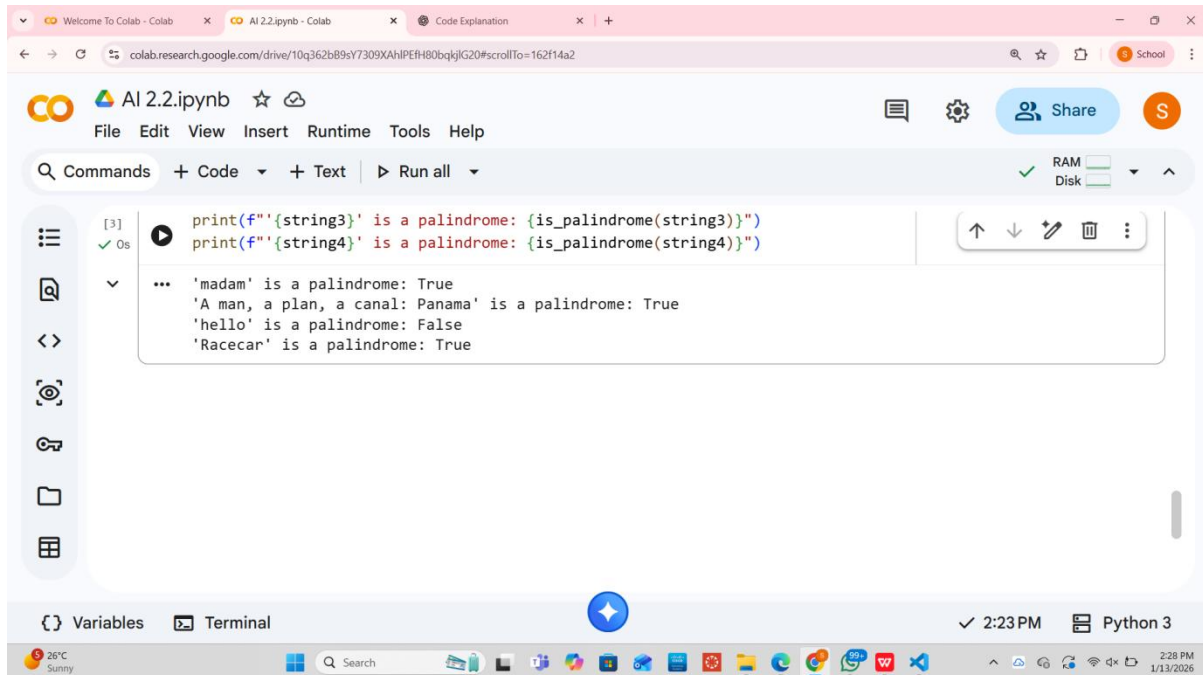


The screenshot shows a VS Code editor with a file named `ai.py`. The code is a Python program that checks if a string is a palindrome. It prompts the user to enter a string, reverses it, and compares the two. If they are equal, it prints that the string is a palindrome; otherwise, it prints that it is not.

```
1 #Write a python program that checks the string is palindrome are not using functions
2 string = input("Enter a string: ")
3 reversed_string = string[::-1]
4 if string == reversed_string:
5
6
7     print(f'{string} is a palindrome.')
8 else:
9     print(f'{string} is not a palindrome.')
10
11
12
13
```

On the right side of the editor, there is a 'CHAT' panel with the text 'Build with Agent' and a prompt 'Describe what to build next'.

Output:
Gemini:



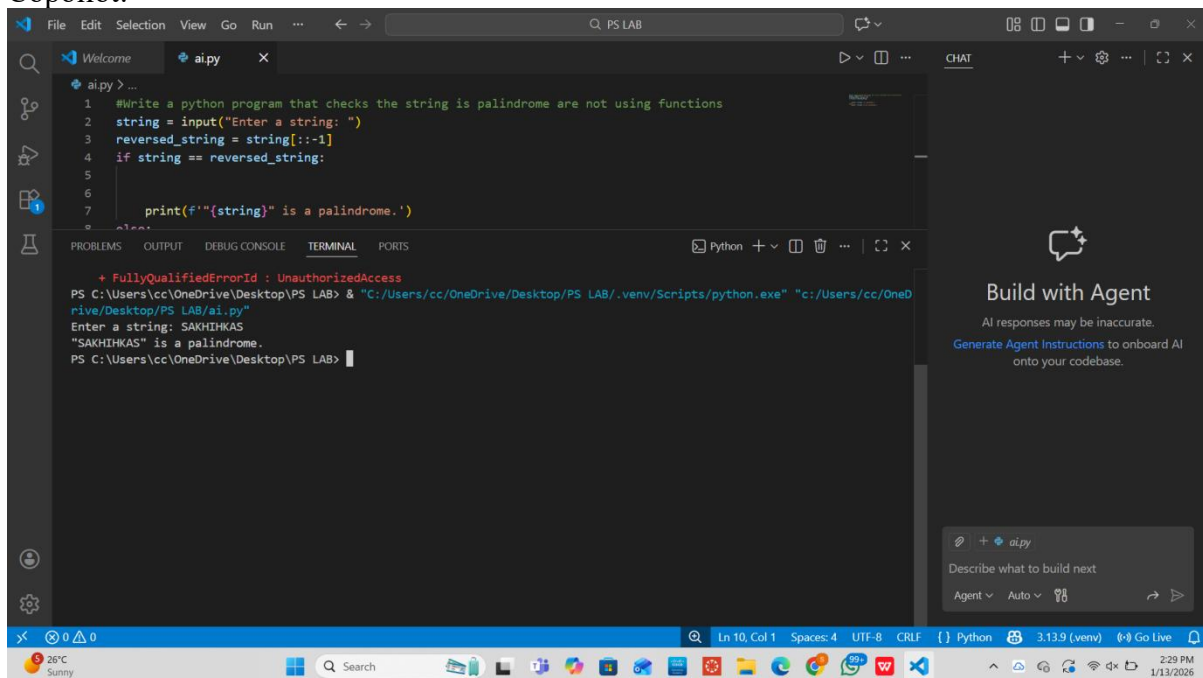
The screenshot shows a Google Colab notebook titled 'AI 2.2.ipynb'. The code cell contains the following Python code:

```
print(f'"{string3}" is a palindrome: {is_palindrome(string3)}')
print(f'"{string4}" is a palindrome: {is_palindrome(string4)}')
```

The output of the code cell is:

```
... 'madam' is a palindrome: True
'A man, a plan, a canal: Panama' is a palindrome: True
'hello' is a palindrome: False
'Racecar' is a palindrome: True
```

Copilot:



The screenshot shows Visual Studio Code with a file named 'ai.py'. The code in the file is:

```
1 #Write a python program that checks the string is palindrome are not using functions
2 string = input("Enter a string: ")
3 reversed_string = string[::-1]
4 if string == reversed_string:
5
6
7     print(f'"{string}" is a palindrome.')
8
```

The terminal window shows the execution of the program:

```
PS C:\Users\cc\OneDrive\Desktop\PS LAB> "C:\Users\cc\OneDrive\Desktop\PS LAB\.venv\Scripts\python.exe" "c:\Users\cc\OneDrive\Desktop\PS LAB\ai.py"
Enter a string: SAKHIHKAS
"SAKHIHKAS" is a palindrome.
PS C:\Users\cc\OneDrive\Desktop\PS LAB>
```

Explantation:

The main difference between the two codes lies in how they handle the input string. The first code removes all non-alphanumeric characters and ignores case, making it more reliable for checking palindromes in real-world sentences. The second code performs a direct comparison without preprocessing, making it simpler but less accurate for strings containing spaces, punctuation, or uppercase letters.

Task4

Prompt: Write a Python function that checks if a string is a palindrome by comparing characters logically.

Explanation:

Let's break it down:

1. `import re:`

- **Purpose:** This line imports the `re` module, which provides regular expression operations. Regular expressions are used here

2. `def is_palindrome_logical(input_string):`

- **Purpose:** This defines a function named `is_palindrome_logical`. It takes one argument, `input_string`, which is the str

3. `cleaned_string = re.sub(r'^a-zA-Z0-9', '', input_string).lower():`

- **Purpose:** This is a crucial step for preparing the string for comparison.
 - `re.sub(r'^a-zA-Z0-9', '', input_string)`: This uses a regular expression to find and replace characters. The letter (`A-Z`), a lowercase letter (`a-z`), or a digit (`0-9`). All such matched characters are replaced with an empty string.
 - `.lower()`: After removing non-alphanumeric characters, this converts the entire resulting string to lowercase. This ensures that characters like 'A' and 'a' are treated as the same character.
 - The result is stored in the `cleaned_string` variable.

4. `left = 0:`

[Compare the two palindrome functions](#)

[Test the palindrome functions with custom inputs](#)

[Explain how to refactor the palindrome functions](#)

Comment:

The explanation clearly describes how the `is_palindrome` function works step by step. Each line of code is explained in an easy and understandable way. The reasons for converting the text to lowercase and removing special characters are clearly mentioned. The explanation of reversing the string using slicing is simple and clear. Overall, the explanation shows a good understanding of how the palindrome logic works in Python.